String

Formatting strings

Proper text formatting makes data easier to read and understand.

Presentation types

It indicates what type is being formatted so Python can determine whether the other formatting information is allowed for that type.

Integers

The `d` presentation type formats integer values as strings. There also are integer presentation types (`b`, `o` and `x` or `X`) that formats the integers using the binary, octal or hexadecimal number systems.

```
num = 10
print(f'{num:d}')    # 10
print(f'{num:b}')    # 1010
print(f'{num:o}')    # 12
print(f'{num:x}')    # a
```

```
print(f'{134.56:d}')
```

- Using integer presentation type on non-integers such as float literal (above) will result in `ValueError`

Characters

The `c` presentation type formats an integer character code as the corresponding character:

```
print(f'{100:c}')    # d → since 100 is character code for 'd'
```

```
print(f'{'a':c}')
```

- Above code will result in `ValueError` as `'a'` is a string literal.

Strings

The `s` presentation type is the default.
If you specify `s` explicitly, the value to format must be a variable that references a string, an expression that produces a string or a string literal.
If you do not specify a presentation type, as in the second placeholder below, non-string values like the integer 7 are converted to strings:

```
msg = 'hello'
print(f'{msg:s}')    # hello
print(f'{1000:s}')   # ValueError
```

Floating-point and decimal values

You've used the `f` presentation type to format floating-point and Decimal values.
For extremely large and small values of these types, Exponential (scientific) notation or `e` or `E` presentation type can be used to format the values more compactly.

```
speed_of_light = 300000000.000

# .2 → two numbers after decimal point
print(f'{speed_of_light:.2f}')    # 300000000.00
print(f'{speed_of_light:.2e}')    # 3.00e+08
```

`replace`

Method `replace` takes two substrings. It searches a string for the substring in its first argument and replaces each occurrence with the substring in its second argument. The method returns a new string containing the results.

```
s = 'i love mango, mango is red in color, an mango a day
keeps the doctors away'
s = s.replace('mango', 'apple')
print(s)
```

```
# output → i love apple, apple is red in color, an apple a day keeps the doctors away
```

split

When you read a sentence, your brain breaks it into individual words, or **tokens**. Tokens typically are separated by whitespace characters such as blank, tab and newline - the separators are known as ***delimiters.***

String method `split` with no arguments tokenizes a string by breaking it into substrings at each whitespace character, then returns a list of tokens.

```
s = 'what a sunny day!'
tokens = s.split()
print(tokens)    # ['what', 'a', 'sunny', 'day!']
```

To tokenize a string at a custom delimiter (such as each comma-and-space pair), specify the delimiter string (such as, ', ')
that split uses to tokenize the string:

```
ingredients = 'bread, cookies, eggs, flour'
tokens = ingredients.split(', ')
print(tokens)    # ['bread', 'cookies', 'eggs', 'flour']
```

If you provide an integer as the second argument, it specifies the maximum number of splits. The last token is the remainder of the string after the maximum number of splits:

```
ingredients = 'bread, cookies, eggs, flour'
tokens = ingredients.split(', ', 2)
print(tokens)    # ['bread', 'cookies', 'eggs, flour']
```

There is also an `rsplit` method that performs the same task as split but processes the maximum number of splits from the end of the string toward the beginning.

```
ingredients = 'bread, cookies, eggs, flour'
tokens = ingredients.rsplit(', ', 2)
```

```
print(tokens)    # ['bread, cookies', 'eggs', 'flour']
```

join

String method `join` concatenates the strings in its argument, which must be an iterable containing only string values; otherwise, a `TypeError` occurs.
The separator between the concatenated items is the string on which you call join.

```
names = ['alice', 'john', 'paul', 'shelly']
names_csv = ','.join(names)
print(names_csv)    # alice,john,paul,shelly
```

Joining the result of a list comprehension

```
nums = '-'.join(str(i**2) for i in range(11))
print(nums)    # 0-1-4-9-16-25-36-49-64-81-100
```

`in` and `not in` operators

If you need to know only whether a string contains a substring, use operator `in` or `not in`:

```
s = 'theres a lady whos sure, all the glitter is gold, and
shes buying a stairway to heaven'

print('Lady' in s)    # False → python is case sensitive!
print('lady' in s)
print('elevator' not in s)    # True
```